TECHNICAL SPECIFICATION & USER JOURNEY DOCUMENT

Smart Campus Entry Monitoring & Biometric Access Control System

Real-time, role-based campus gate monitoring with fingerprint-terminal integration

Source code: https://github.com/Sanket1516/Smart-Campus-Monitoring-System

Prepared by: Sanket Mali (Roll No. E265), B.Tech Computer Science
Institution: SVKM's NMIMS MPSTME, Shirpur
Guide: Dr. Nitin Choubey
Date: 31 October 2026

1 PROJECT OVERVIEW & FUNCTIONAL SPECIFICATION

1.1 Executive summary & problem statement

Smart Campus Entry Monitoring is a MERN web application that replaces the handwritten gate register. It records every student entry and exit automatically and shows it live. A fingerprint terminal (eSSL X990) sends a numeric user ID to the backend. The backend decides whether it is an entry or an exit, applies blocks and warden approvals, flags curfew violations, emails or texts the parents, and pushes the event to the dashboards. Guards can also type a SAP ID when a terminal is unavailable. The repository holds a working version: React frontend, Express API, MongoDB, Docker and a CI/CD pipeline.

Problem (current practice)	Impact	Proposed solution (implemented)
Manual paper gate registers, guard relies on visual recognition	Errors, falsification, unregistered entry	Biometric scan → automated, timestamped EntryLog. Unauthorized IDs logged and alerted
No live view of who is inside, no hosteller exit control	Curfew violations found late	Live dashboard, per-hostel curfew flag, warden-approved time-bound exit windows
Parents not informed, no analytics or audit trail	No data-driven decisions	Async e-mail/SMS to parents, hourly/7-day analytics, AuditLog of admin actions

1.2 Objectives

- Automate gate entry/exit recording with a smart toggle (entry → exit → re-entry) from biometric terminals, with SAP-ID fallback.
- Apply access rules at the gate: blocked students, unknown terminals or IDs, and warden approval for hostellers.
- Show each role only what it needs (admin, security, warden), plus a small public page, and keep an audit trail of changes.
- Show live campus status, give analytics, notify parents, and package everything in containers with automated deployment.

1.3 Target users and stakeholders

Role	Access (verified in code)	Primary needs
Admin	All modules: students, enrollment, access control, terminals, settings, staff, audit	Configure and govern the system
Security	Dashboard, Scanner, Logs, alerts, terminal status, block list	Fast gate decisions, visitor entry
Warden	Hostel-scoped dashboard, logs, analytics, hostellers, Warden Portal	Approve/reject exit requests, track late returns
Hosteller (student)	Public /student/exit-request (name + phone check)	Request leave and see status
Public / Parents	/live read-only page, parents receive e-mail/SMS only (no parent login)	Awareness of campus/child movement

1.4 FUNCTIONAL REQUIREMENTS (WHAT THE SYSTEM DOES)

ID	Requirement	Evidence	Status
FR1	Accept biometric scan events, authenticate terminal by machineNumber + deviceSN, return allow true/false	POST /api/fingerprint/scan	Implemented
FR2	Manual SAP-ID entry/exit with unauthorized-attempt logging, visitor entry capture	POST /api/scan, /api/visitors	Implemented
FR3	Auto toggle entry / exit / re-entry from latest EntryLog state	fingerprint & scan controllers	Implemented
FR4	Block/unblock students with reason, log blocked attempts and raise alerts	/api/access, AccessControlLog	Implemented
FR5	Student CRUD (soft delete), search/filter, Excel (.xlsx/.xls) bulk upload with row-level errors	/api/students, multer + xlsx	Implemented
FR6	Hosteller exit request → warden approval → time-limited access window, 5-min expiry job	/api/hosteller, node-cron	Implemented
FR7	Curfew late-return flag (default 22:00, per-hostel override) and warden late-return alert	configService.isPastCurfew	Implemented
FR8	Terminal registry, heartbeat, offline detection (2 min), enrollment-station mode	/api/terminals, terminalJobs	Implemented
FR9	Live events (scan:live, blocked, unauthorized, warden, terminal) over Socket.IO	socket.js, socketService	Implemented
FR10	Dashboard, hourly distribution, 7-day trend, hosteller status, public live page	/api/dashboard, /api/live	Implemented
FR11	Parent e-mail (SMTP) and SMS (Fast2SMS) notifications, toggles in settings	notification.js	Implemented*
FR12	Staff accounts, hostels, system settings, maintenance mode, audit log	/api/auth, /settings, AuditLog	Implemented
FR13	Dashboard chat: 5 fixed intents (Ollama gemma:2b + keyword fallback) answered from DB	/api/ai-query	Implemented (limited)
FR14	Parent login portal, multi-gate campus-wide rollout (10,000+ students)	Proposal only	Proposed

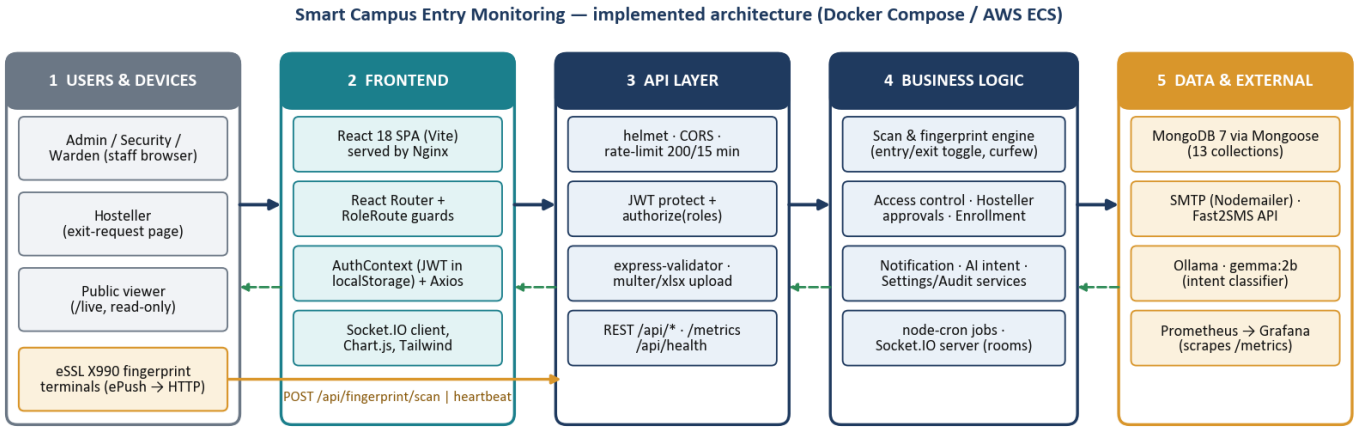
**Delivery depends on SMTP / Fast2SMS credentials supplied through environment variables.*

1.5 NON-FUNCTIONAL REQUIREMENTS (HOW WELL IT OPERATES)

Category	Requirement / design provision	Status
Security	JWT + bcrypt(12), role guards, helmet, CORS, rate limit, validation (see page 5)	Implemented
Performance / real-time	Terminal reply is a single JSON round trip, events pushed via WebSocket, dashboards poll every 30 s	Implemented, not load-tested
Reliability	Containers set to restart unless-stopped. Mongo health check. Offline monitor. Async notifications never block a scan	Implemented
Usability / compatibility	Responsive Tailwind UI, role-aware navigation, toast + sound feedback. Asia/Kolkata time zone	Implemented
Maintainability / observability	Layered routes→controllers→services→models. Prometheus /metrics, Grafana, /api/health	Implemented
Scalability	Stateless API behind Nginx, single MongoDB node, Socket.IO has no shared adapter	Partial

2 TECHNICAL ARCHITECTURE & TECHNOLOGY SPECIFICATIONS

2.1 System architecture



Solid arrows: request path (user → frontend → API → business logic → MongoDB / external service). Dashed: JSON response to the user. Socket.IO pushes scan:live, scan:blocked, scan:unauthorized, terminal:offline … to all connected clients.

Terminals receive only {"allow": true|false}. Fingerprint matching runs inside the device and the server receives a numeric user ID.

2.2 Component responsibilities

Layer	Architecture (as implemented)
Frontend	React 18 + Vite SPA served by Nginx. React Router with PrivateRoute/RoleRoute guards driven by rolePermissions.js. AuthContext keeps the JWT in localStorage. An Axios interceptor adds the Bearer token and logs out on any 401. SocketContext subscribes to live events. Chart.js renders analytics.
Backend	Node 20 / Express 4 in route → controller → service → model layers. 17 route groups under /api, global error handler (validation, duplicate-key, JWT errors), Prometheus metrics middleware, HTTP + Socket.IO server on port 5000.
Database	MongoDB 7 via Mongoose 8, 13 collections: Admin, Student, Hostel, EntryLog, UnauthorizedLog, VisitorEntry, TerminalConfig, DeviceAllocation, HostellerRequest, AccessControlLog, AlertLog, AuditLog, SystemConfig. Entry records use a YYYY-MM-DD IST date key plus entered/exited status.
AI component	Ollama (gemma:2b) is not used to write answers. It only classifies a question into one of 5 intents (total_inside, late_returns_today, hostellers_outside, unauthorized_today, unknown). Answers are computed from MongoDB. On timeout/error a keyword classifier takes over.
Communication	REST/JSON over HTTP (browser, terminals). WebSocket via Socket.IO for push (warden events go to room warden:<id>). SMTP and HTTPS to Fast2SMS. HTTP to Ollama. Prometheus scrapes /metrics.

2.3 Technology specification

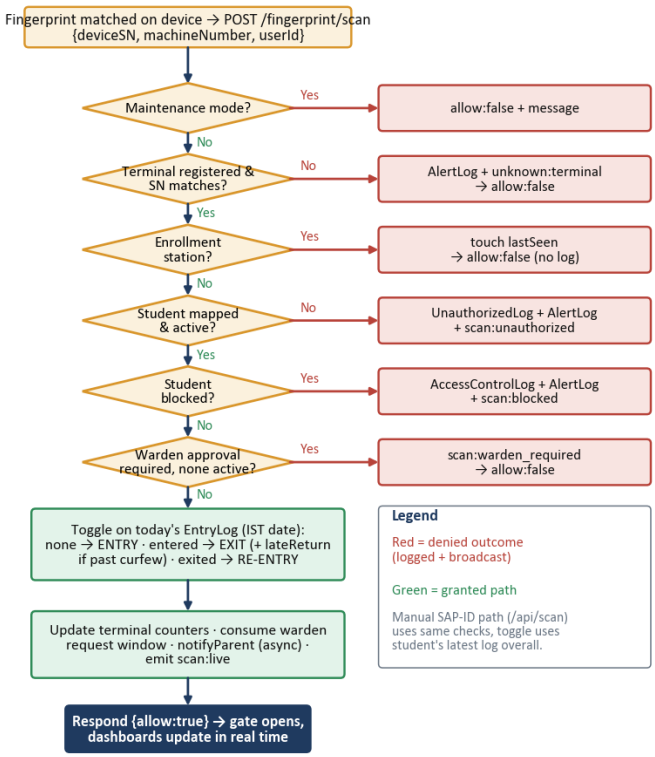
Component	Technology	Purpose	Implementation details
UI framework	React 18.3, Vite 5, Tailwind 3	Single-page role-based console	Pages: Dashboard, Scanner, Logs, Analytics, Hostellers, Students, Enrollment, Access, Warden Portal, Terminals, Settings, Live, Exit Request
Client libs	Axios, Chart.js 4, socket.io-client, react-hot-toast, html5-qrcode, tesseract.js	HTTP, charts, live updates, feedback, barcode / OCR input	Token interceptor, alert sounds (3 .wav files), BarcodeScanner and OcrScanner components on the Scanner page
API server	Node 20, Express 4.21	REST API and routing	17 route files, controllers, services, CORS limited to CLIENT_URL
Auth & security	jsonwebtoken, bcryptjs, helmet, express-rate-limit, express-validator	Identity, access, hardening	JWT 7 d default, bcrypt 12 rounds, 200 req / 15 min on /api, validators on write routes
Database	MongoDB 7, Mongoose 8	Persistent document store	Schema validation, select:false on password / fingerprintTemplate
Real-time	Socket.IO 4.8	Push alerts and live ticker	JWT handshake, 11 named events, warden rooms
Biometric link	eSSL X990 (ePush), node-zklib	Scan source and device user sync	Terminal posts {deviceSN, machineNumber, userId}. DeviceAllocation maps terminal-local ID to student
Bulk import	multer 2, xlsx 0.18	Excel enrollment	Row validation, valid rows inserted, failures returned with reasons
Scheduling	node-cron 4	Background jobs (IST)	Expire approvals every 5 min, reset scansToday at 00:00, offline monitor every 60 s
Notifications	Nodemailer 6, Fast2SMS REST	Parent / warden alerts	SMTP via env vars, SMS skipped if API key absent
AI intent	Ollama + gemma:2b	Chat intent classification	AI_TIMEOUT_MS bound, deterministic fallback
Ops	Docker Compose, Nginx, Prometheus, Grafana, GitHub Actions, AWS ECR/ECS	Packaging, monitoring, delivery	6 services in compose, prom-client metrics, see page 5

3 SYSTEM WORKFLOW, MODULES & DATA FLOW

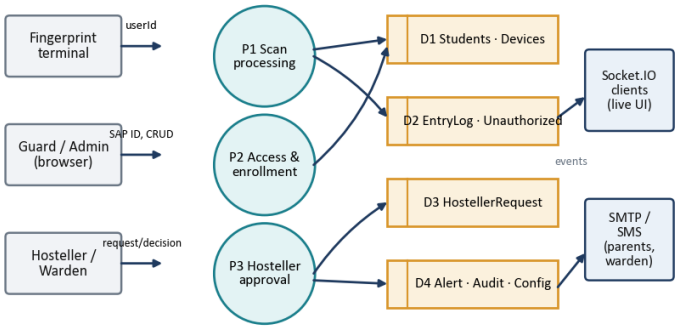
3.1 Module specification

Module	Purpose	Input	Processing	Output	Dependencies
M1 Auth & RBAC	Staff identity, roles	username, password, Bearer JWT	bcrypt compare, JWT sign/verify, active-flag check on every request, authorize(roles)	Token, admin profile, 401/403	Admin model, auditService
M2 Scan & Biometric	Entry/exit decision	userId + terminal IDs, or SAP ID	Workflow below: terminal, student, block, warden checks. Toggle. Curfew flag	allow flag / JSON result, EntryLog, alerts	M3, M4, M5, M6, M8, configService
M3 Access Control	Block / unblock	SAP ID, reason	Update accessStatus, write AccessControlLog + audit	Block list, log	Student, AuditLog
M4 Student & Enrollment	Student lifecycle, Excel import	Form or .xlsx file	Validate, normalise category/hostel, insertMany, soft delete	Counts + row errors	Student, Hostel, DeviceAllocation
M5 Hosteller Approval	Leave request workflow	Name + phone, reason, return time	Create → approve/reject → accessValidUntil → cron expiry	Request status, alerts	HostellerRequest, M6, M8
M6 Real-time Hub	Live push	Domain events	Socket.IO emit, warden room targeting	scan:*, terminal:*, warden:* events	socket.js
M7 Terminal Mgmt	Device registry & health	Heartbeat, CRUD	Mark online/last-seen, offline job, daily counter reset	Status, offline events	TerminalConfig, cron
M8 Notification	Parent / warden messages	Student, action, time	Build HTML/text, send e-mail and SMS asynchronously	Delivery result (logged)	Nodemailer, Fast2SMS, SystemConfig
M9 Analytics, Live, AI	Statistics and queries	Date, filters, chat text	Aggregate EntryLog, classify intent, hostel-scoped for wardens	Charts, counts, answers	EntryLog, Ollama

End-to-end biometric scan workflow

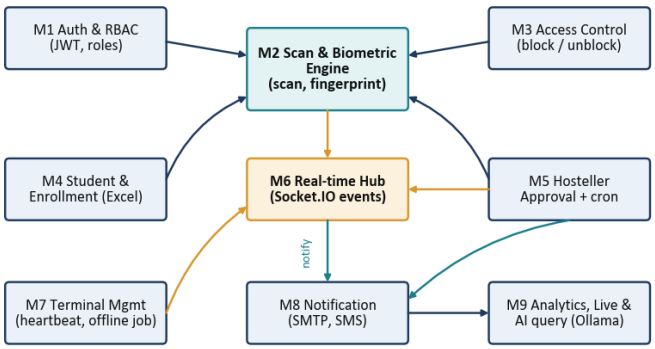


Data-flow diagram (level 1)



Reports/dashboards read D1–D4 (read-only aggregation) and render Chart.js views.

Module interaction diagram



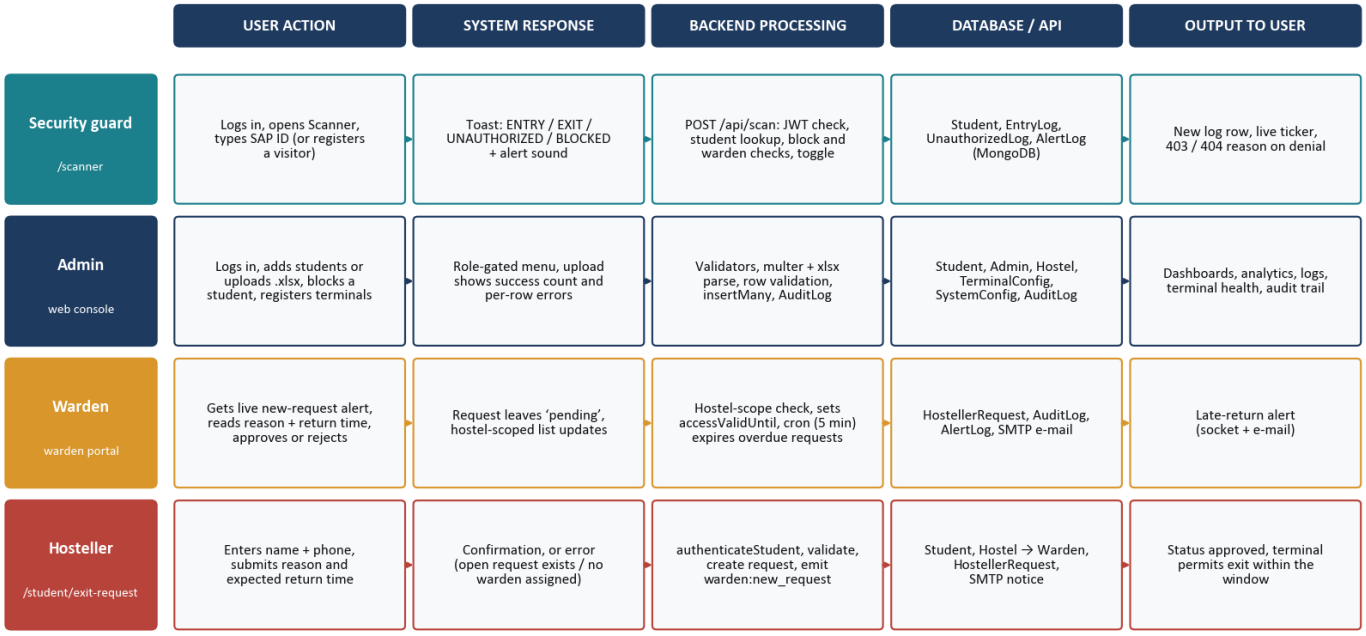
Black = calls/reads · Amber = emits event · Teal = notification dispatch. All modules persist via Mongoose models.

3.2 Cross-cutting mechanisms

Concern	How it works in the implementation
API communication	Axios sends JSON with Authorization: Bearer <JWT>. Terminals call /api/fingerprint/scan and /heartbeat without a JWT (identified by machineNumber + deviceSN) and receive {allow:true|false}.
Database interaction	Controllers use Mongoose queries. EntryLog is looked up by sapId + IST date. populate() joins Student→Hostel→Warden. Scoped queries restrict wardens to their hostel's SAP IDs.
Auth & authorization	protect() verifies JWT, loads Admin, rejects missing or deactivated accounts. authorize() checks role. React RoleRoute mirrors the same matrix. Hosteller requests use a separate name + phone check (authenticateStudent).
Error handling	Central errorHandler maps Mongoose ValidationError / duplicate key (400) and JWT errors (401). Controllers return 4xx with a message. The terminal endpoint never throws – any failure returns allow:false. Notification and socket failures are caught and logged.
External services	Fire-and-forget calls: notifyParent().catch(log). Ollama call aborted after AI_TIMEOUT_MS then keyword fallback. SMS skipped when no API key. Maintenance mode short-circuits scans with a message.

4 USER JOURNEY & INTERACTION DESIGN

4.1 Journey infographic: user action → system response → backend → database/API → output

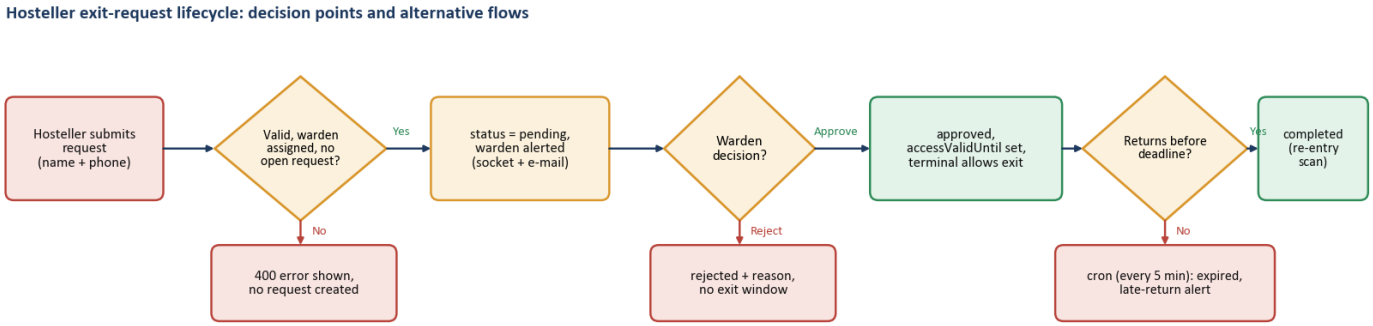


Public viewer (no login): /live → GET /api/live → read-only counts, branding and terminal status. Biometric entries follow the workflow on page 3.

4.2 Role journeys: entry point, outcomes and decisions

Role	Entry & login	Landing	Primary actions	Success	Failure / exception
Security guard	/login (username + password) → JWT	Dashboard: counts, charts, live ticker	Scanner: enter SAP ID, or terminal scans automatically. Register visitor. Review Logs	Entry/exit logged, parent notified, sound cue	Unknown ID → UnauthorizedLog + alert, blocked → 403 + alert, warden approval missing → 403
Admin	/login	Dashboard + admin menu	Add/upload students, block, register terminals, staff and hostels, change settings	Records saved, audit entry	Duplicate SAP ID, bad department/course, invalid row → per-row error list. 403 for other roles
Warden	/login (scoped to own hostel)	Dashboard of hostel only	Warden Portal: approve/reject, Hostellers page: who is outside / late	Student can exit within window	Request outside own hostel → 403, student overdue → expiry + late alert
Hosteller	/student/exit-request (name + phone)	Request form + latest status	Submit reason, return time, check status	Request pending → approved	Wrong credentials 401, open request exists, no warden assigned (400)
Public viewer	/live (no login)	Read-only campus status	View counts, terminal status	Page renders	API down → empty state

4.3 Hosteller exit-request lifecycle (key decision points)



4.4 Alternative flows, error scenarios and UX considerations

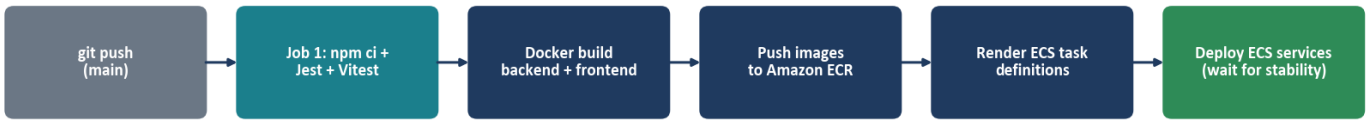
- **Alternative flows:** terminal unavailable → guard uses SAP-ID scanner (barcode/OCR helpers exist). Student without warden-approval flag skips the approval gate. Re-entry after exit creates a new EntryLog row.
- **Exceptions:** expired/invalid JWT or deactivated account → 401 and automatic redirect to /login. Unknown terminal → alert + unknown:terminal event. Maintenance mode → scans refused with a message. Terminal silent for 2 min → marked offline and broadcast.
- **Decision points:** terminal known? student mapped? blocked? warden approval active? last status entered or exited? past curfew?
- **UX considerations:** role-filtered navigation, mobile drawer, toast/sound feedback, live status banner, server-side pagination, 30-second auto-refresh, public page needing no sign-in.

5 SECURITY, DEPLOYMENT, TESTING & FUTURE SCOPE

5.A Security specifications

Area	Implemented measure	Observed gap / note
Authentication	JWT (7 d default, JWT_SECRET env). Bcrypt cost 12. Password select:false. Deactivated accounts rejected on every request and socket handshake	No refresh/revocation, token kept in localStorage (XSS exposure)
Authorization	authorize(roles) on routes. Client RoleRoute. Warden queries scoped to assigned hostel. Audit log on staff, student, access, settings changes	POST /api/scan requires login but no role check
Data protection	Soft delete. Fingerprint template field select:false and AES-encrypted with FP_SECRET key. Terminals send only numeric IDs	Key falls back to a default if FP_SECRET unset, confirm no templates are persisted in production
Input validation	express-validator on terminal, auth, student, hostel and request routes. ISO-8601 / Mongo-ID checks. Mongoose schema enums	Student-name regex lookup is not escaped
API security	helmet, CORS to CLIENT_URL, 200 req/15 min limit on /api, centralised error handler	Terminal endpoints have no shared secret/HMAC, limit also applies to terminals
Sensitive data	.env files for secrets, .env.example only with placeholders	studentAuth logs the username and first characters of the password to the console

5.B Deployment architecture



- **Hosting:** Docker Compose runs mongo:7, ollama, backend (node:20-alpine, TZ Asia/Kolkata), frontend (Nginx proxying /api and /socket.io), Prometheus and Grafana. AWS target: images in Amazon ECR, services on ECS cluster smart-campus-cluster (us-east-1).
- **CI/CD (implemented):** .github/workflows/deploy.yml on push to main → test job (Node 20, npm ci, Jest, Vitest) → build/push backend and frontend images → render and deploy ECS task definitions with stability wait. Secrets come from GitHub Actions.
- **External dependencies:** MongoDB, SMTP provider, Fast2SMS, Ollama model, eSSL terminals on the LAN. The Cloudflare-Tunnel pilot and Amazon SES in the proposal are **proposed**, not present in code.

5.C Testing and validation

Type	Completed (verified by running on 31 Oct 2026)	Recommended (not yet done)
Unit / API	Backend Jest + Supertest: **7 tests passed** (login valid / invalid / missing fields, scan authorised and unknown SAP ID, student lookup found / not found)	Fingerprint scan, toggle, curfew and approval-window unit tests
Frontend	Vitest + Testing Library: **2 tests passed** (Login render, StatCard)	Role routing, scanner and upload component tests
Integration / system	TESTING_CHECKLIST.md defines manual cases for auth, navigation and modules. Execution results are not recorded	Execute checklist with a real terminal, end-to-end tests
Performance / security	None available, no load figures are claimed	Load test (scan burst), dependency audit, penetration test

5.D Limitations

- API.md documents an older contract (e-mail login, role “guard”). Code uses username and admin/security/warden.
- Manual path uses a UTC date and the latest log overall. Terminal path uses IST date and today’s log: edge cases near midnight may differ.
- Hosteller verification is name + phone, names are not unique keys.
- Single database node, no Socket.IO adapter. 10,000-student scale unverified.
- Chat covers five intents only, notification delivery needs external credentials.

5.E Future enhancements

- **Scalability:** MongoDB replica set, Redis adapter for Socket.IO, multi-gate rollout, ECS autoscaling.
- **Performance:** indexes on date/sapId, cached dashboard aggregates, shared date helper.
- **Integrations:** parent portal / mobile app, WhatsApp or SES delivery, ERP/SAP sync.
- **UX:** richer analytics export, multilingual UI, configurable alert rules.
- **Security:** HMAC-signed terminal calls, httpOnly cookies + refresh tokens, 2FA for admins, automated test suite in CI gates.

5.F Assumptions and missing information

Item	Assumption / gap
Terminal hardware	eSSL X990 behaviour (ePush, local matching) is taken from the proposal. No device was exercised for this document.
Fingerprint templates	The schema holds an encrypted template field. Whether templates are stored server-side in production cannot be verified from the repository.
Cloud environment	AWS task definitions, secrets and live URLs are not in the repository. The deployment is described from the workflow and compose files.
Screenshots / scale	UI described from source code (College Project Images.docx not used as a source). The 10,000+ student target is a proposal figure, not measured.

Technical conclusion. Most of what the proposal describes is already built and runs: biometric scans, role-based access, warden approvals, live events, notifications, analytics, Docker deployment and a CI/CD pipeline. The college gets a searchable, timestamped gate record in place of the paper register. Before wider use, the security gaps listed in 5.A should be fixed and load and end-to-end tests should be run.